

Spécification Technique Complète – Backend WEZON

Agrégateur IA d'Actualités Africaines + Chatbot RAG

1. Vision Technique Globale

WEZON est une plateforme d'agrégation, de structuration, de vérification et de consultation de l'actualité africaine.

Le backend doit permettre :

1. Ingestion massive et continue d'informations provenant :

- Flux RSS
- Sites web sans RSS (scraping contrôlé)
- Agents de recherche web automatisée

2. Traitement intelligent via différents agents IA :

- Nettoyage, normalisation, extraction de contenu
- Détection de langue et traduction
- Classification thématique
- Extraction d'entités (NER)
- Vérification de fiabilité
- Génération de résumés

3. Indexation et stockage :

- Base relationnelle structurée (PostgreSQL)
- Vector Store pour RAG

4. API haut niveau :

- Consultation d'articles
- Recherche
- Accès aux résumés
- Chatbot avec RAG + recherche web

5. Orchestration robuste :

- Scheduling des tâches
- Monitoring
- Gestion des erreurs

2. Architecture Technique

2.1. Microservices / Agents

WEZON est composé d'agents indépendants exécutés via Celery.

Agents :

- **RSS Ingestion Agent**
- **Web Scraping Agent**
- **Content Normalization Agent**
- **Language Detection Agent**
- **Translation Agent**
- **Classification & NER Agent**
- **Fact-checking Agent**
- **Article Summarizer**
- **Daily Summary Generator**
- **Embedding Generator**
- **RAG Retriever**
- **Chat Orchestrator**

Tous communiquent via :

- PostgreSQL
- Redis (queue)
- Vector Store

2.2. API Backend

Expose les endpoints pour :

- Articles
- Sources
- Résumés
- Chatbot
- Admin

Framework recommandé : ****FastAPI****

3. Schéma de Données (PostgreSQL)

Table: countries

- id (PK)
- name
- iso
- region

Table: sources

- id
- name
- url
- type (rss, web, api)
- country_id
- language_id
- trust_score
- fetch_frequency (seconds)

Table: articles

- id
- source_id

- country_id
- language_id
- title
- content
- summary
- url
- published_at
- fetched_at
- trust_score
- raw_html
- processing_status (pending, processed, failed)

Table: article_categories

- id
- article_id
- category

Table: article_entities

- id
- article_id
- type (person, org, location, keyword)
- value

Table: article_translations

- id
- article_id
- language_id
- translated_title
- translated_content
- translated_summary

Table: daily_summaries

- id
- date
- scope_type
- scope_id
- language_id
- summary_text

4. Pipeline Complet (Flow Technique)

1 — Ingestion

- L'agent lit les flux RSS / scrape les pages
- Stocke le raw HTML + metadata dans PostgreSQL

2 — Normalisation

- Extraction du contenu principal (readability)
- Nettoyage (HTML → texte brut)

3 — Détection Langue

- fastText / langdetect
- Affectation d'une langue interne

4 — Traduction (optionnel)

- Traduction vers FR et EN pour homogénéiser la base

5 — Classification

- Catégorie unique ou multiple :
- politique, économie, tech, sécurité, société, culture, sport, énergies...
- NER (entités) pour enrichir la recherche

6 — Vérification

- Heuristique avancée :
- Cross-matching d'articles
- Score de fiabilité du média
- Cohérence interne

7 — Résumé

- Résumé LLM (court + long)
- Résumé traduit

8 — Embedding

- Embeddings(title)
- Embeddings(summary)
- Embeddings(content)

Stockage dans vector DB

9 — Résumés quotidiens

- Génération automatique chaque soir
- Stockage dans daily_summaries

5. RAG – Fonctionnement Technique

5.1. Indexation

Pour chaque article :

- embeddings générés
- `vector_store.insert({`
`article_id`
`metadata: (country, language, categories, trust_score)`
`})`

5.2. Pipeline Chat

1. Analyse de la question
2. Construction d'une requête vectorielle
3. Récupération des K documents pertinents
4. Recherche web externe si :
 - évènements récents (<1h)
 - faible pertinence interne
5. Fusion des résultats
6. Génération de réponse :
 - réponse structurée
 - sources citées
 - liens externes

6. Endpoints API (FastAPI)

GET /articles

Params :

- country
- category
- language
- search
- date_from / date_to

GET /articles/{id}

GET /daily-summaries

Params :

- date
- scope_type

- scope_id

- language

POST /chat

Body :

- question

- language

- filters: {country, region, time_range}

Admin :

- POST /sources

- GET /sources

- PUT /sources/{id}

7. Orchestration

Scheduling

Utilisation de Celery Beat :

- rss_fetch_task : toutes les X minutes

- web_fetch_task : toutes les X minutes

- daily_summary_task : 23h59

- embedding_task : toute nouvelle insertion

Gestion des erreurs

Stocker :

- erreurs ingestion

- erreurs parsing

- erreurs de classification

- redémarrage automatique

8. Déploiement (Docker Compose)

Services recommandés :

- api
- worker_ingestion
- worker_ai
- redis
- postgres
- qdrant / pgvector
- traefik/nginx

9. Logs & Monitoring

- Grafana + Prometheus
- Loki pour les logs
- Alertes : ingestion bloquée, worker offline, API lente

10. Sécurité

- HTTPS
- JWT pour Admin
- Protection anti-spam pour le chat
- Limitation du scraping conformément aux robots.txt